

Tecnológico de Monterrey

Campus Monterrey

Evidencia2

Diego Villanueva Fernandez
A01199495

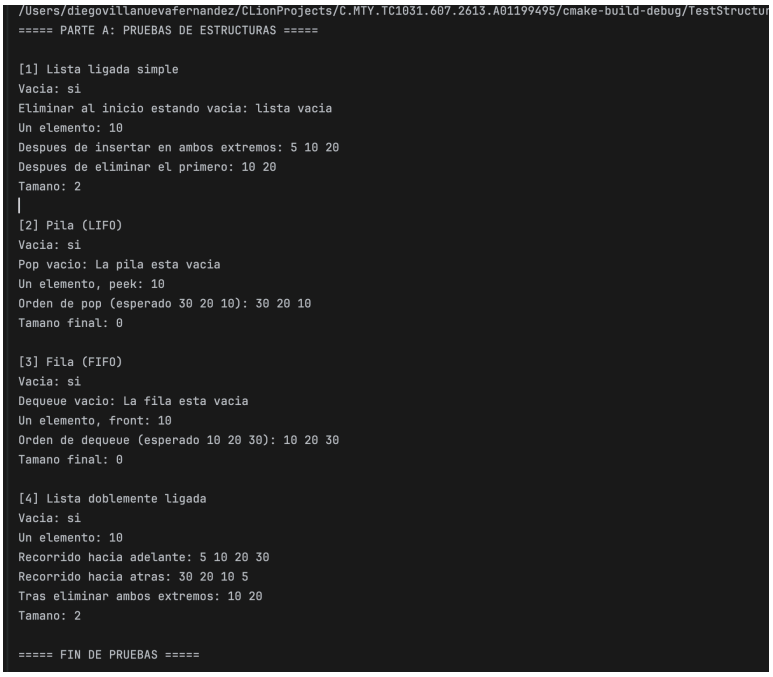
Programación de estructuras de datos y algoritmos fundamentales
Grupo 607

18 de octubre del 2026

Tecnológico de Monterrey

Evidencia de ejecución

Parte A: estructuras y lectura inicial



```
/Users/diegovillanuevafernandez/CLionProjects/C.MTY.TC1031.607.2613.A01199495/cmake-build-debug/TestStructur
===== PARTE A: PRUEBAS DE ESTRUCTURAS =====

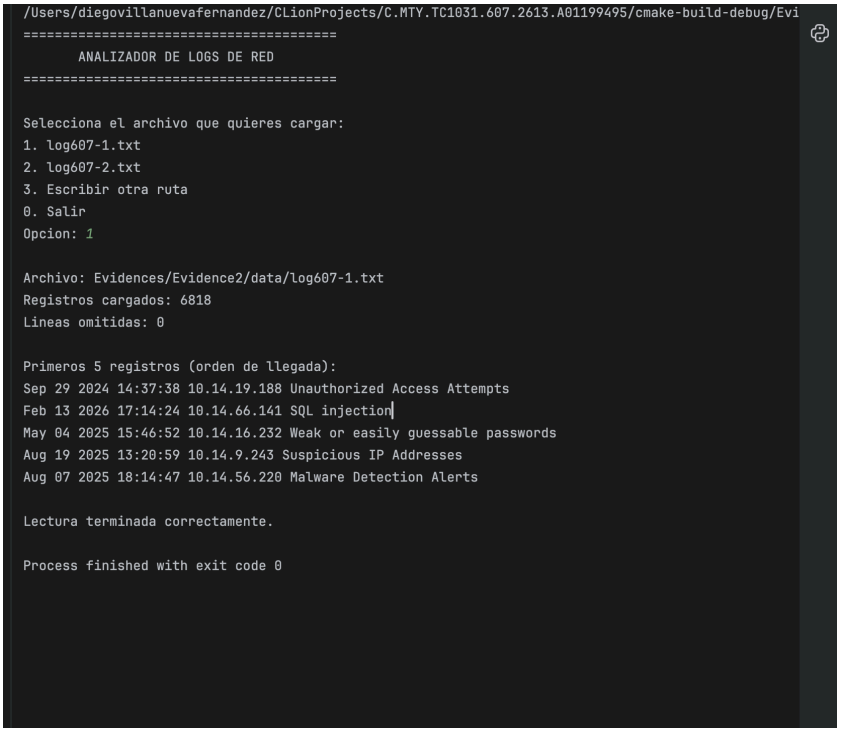
[1] Lista ligada simple
Vacia: si
Eliminar al inicio estando vacia: lista vacia
Un elemento: 10
Despues de insertar en ambos extremos: 5 10 20
Despues de eliminar el primero: 10 20
Tamano: 2

[2] Pila (LIFO)
Vacia: si
Pop vacio: La pila esta vacia
Un elemento, peek: 10
Orden de pop (esperado 30 20 10): 30 20 10
Tamano final: 0

[3] Fila (FIFO)
Vacia: si
Dequeue vacio: La fila esta vacia
Un elemento, front: 10
Orden de dequeue (esperado 10 20 30): 10 20 30
Tamano final: 0

[4] Lista doblemente ligada
Vacia: si
Un elemento: 10
Recorrido hacia adelante: 5 10 20 30
Recorrido hacia atras: 30 20 10 5
Tras eliminar ambos extremos: 10 20
Tamano: 2

===== FIN DE PRUEBAS =====
```



```
/Users/diegovillanuevafernandez/CLionProjects/C.MTY.TC1031.607.2613.A01199495/cmake-build-debug/Evi
========================================
       ANALIZADOR DE LOGS DE RED
========================================

Selecciona el archivo que quieres cargar:
1. log607-1.txt
2. log607-2.txt
3. Escribir otra ruta
0. Salir
Opcion: 1

Archivo: Evidences/Evidence2/data/log607-1.txt
Registros cargados: 6818
Lineas omitidas: 0

Primeros 5 registros (orden de llegada):
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts
Feb 13 2026 17:14:24 10.14.66.141 SQL injection
May 04 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
Aug 19 2025 13:20:59 10.14.9.243 Suspicious IP Addresses
Aug 07 2025 18:14:47 10.14.56.220 Malware Detection Alerts

Lectura terminada correctamente.

Process finished with exit code 0
```

Tecnológico de Monterrey

Lectura de Logs y primeras mediciones

```
/Users/diegovillanuevafernandez/CLionProjects/C.MTY.TC1031.607.2613.A01199495/cm
========================================
       ANALIZADOR DE LOGS DE RED
========================================

Selecciona el archivo que quieres cargar:
1. log607-1.txt
2. log607-2.txt
3. Escribir otra ruta
0. Salir
Opcion: 1

Archivo: Evidences/Evidence2/data/log607-1.txt
Registros cargados: 6818
Lineas omitidas: 0

Primeros 5 registros (orden de llegada):
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts
Feb 13 2026 17:14:24 10.14.66.141 SQL injection
May 04 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
Aug 19 2025 13:20:59 10.14.9.243 Suspicious IP Addresses
Aug 07 2025 18:14:47 10.14.56.220 Malware Detection Alerts

Lectura terminada correctamente.
```

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia (proximamente)
3. Mostrar por dia en orden descendente (proximamente)
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 1

Que crecimiento de comparaciones esperas?
1. n log n
2. n^2
Prediccion: 1

========== RESULTADOS DEL ORDENAMIENTO ==========
Algoritmo: Merge Sort (estable)
Archivo: Evidences/Evidence2/data/log607-1.txt
Registros: 6818
Tiempo: 2.915 ms
Comparaciones: 78330
Profundidad maxima de recursion: 14
Prediccion: n log n
Coincidio con el crecimiento teorico del algoritmo: Si
Orden cronologico correcto: Si
Archivo generado: Evidences/Evidence2/out/output608.out
```

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia (proximamente)
3. Mostrar por dia en orden descendente (proximamente)
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 1

La lista ya esta ordenada cronologicamente.
Archivo generado: Evidences/Evidence2/out/output608.out
```

Ejecución del segundo archivo

```
========================================
               MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia (proximamente)
3. Mostrar por dia en orden descendente (proximamente)
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 5
Hasta luego.

Process finished with exit code 0
```

```
/Users/diegovillanuevafernandez/CLionProjects/C.MTY.TC1031.607.2613.A01199495/cmake-build-
========================================
      ANALIZADOR DE LOGS DE RED
========================================

Selecciona el archivo que quieres cargar:
1. log607-1.txt
2. log607-2.txt
3. Escribir otra ruta
0. Salir
Opcion: 2

Archivo: Evidences/Evidence2/data/log607-2.txt
Registros cargados: 6818
Lineas omitidas: 0

Primeros 5 registros (orden de llegada):
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
Sep 09 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
Sep 08 2024 10:42:42 10.14.40.195 Data corruption

Lectura terminada correctamente.
```

```
========================================
               MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia (proximamente)
3. Mostrar por dia en orden descendente (proximamente)
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 1

Que crecimiento de comparaciones esperas?
1. n log n
2. n^2
Prediccion: 1

========== RESULTADOS DEL ORDENAMIENTO ==========
Algoritmo: Merge Sort (estable)
Archivo: Evidences/Evidence2/data/log607-2.txt
Registros: 6818
Tiempo: 1.911 ms
Comparaciones: 50624
Profundidad maxima de recursion: 14
Prediccion: n log n
Coincidio con el crecimiento teorico del algoritmo: Si
Orden cronologico correcto: Si
Archivo generado: Evidences/Evidence2/out/output608.out
```

Lectura y validación del flujo de ordenamiento

```
/Users/diegovillanuevafernandez/CLionProjects/C.MTY.TC1031.607.2613.A01199495/cmake-build-(
========================================
       ANALIZADOR DE LOGS DE RED
========================================

Selecciona el archivo que quieres cargar:
1. log607-1.txt
2. log607-2.txt
3. Escribir otra ruta
0. Salir
Opcion: 1

Archivo: Evidences/Evidence2/data/log607-1.txt
Registros cargados: 6818
Lineas omitidas: 0

Primeros 5 registros (orden de llegada):
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts
Feb 13 2026 17:14:24 10.14.66.141 SQL injection
May 04 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
Aug 19 2025 13:20:59 10.14.9.243 Suspicious IP Addresses
Aug 07 2025 18:14:47 10.14.56.220 Malware Detection Alerts

Lectura terminada correctamente.
```

```
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 2
Primero debes ordenar los logs con la opcion 1.

========================================
               MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 1

Que crecimiento de comparaciones esperas?
1. n log n
2. n^2
Prediccion: 1

========== RESULTADOS DEL ORDENAMIENTO ==========
Algoritmo: Merge Sort (estable)
Archivo: Evidences/Evidence2/data/log607-1.txt
Registros: 6818
Tiempo: 3.319 ms
Comparaciones: 78330
Profundidad maxima de recursion: 14
Prediccion: n log n
Coincidio con el crecimiento teorico del algoritmo: Si
Orden cronologico correcto: Si
Archivo generado: Evidences/Evidence2/out/output608.out
```

Tecnológico de Monterrey

Búsqueda por día y visualización decreciente

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 2
Ingresa el dia (ejemplo: Sep 14 2024): Sep 14 2024

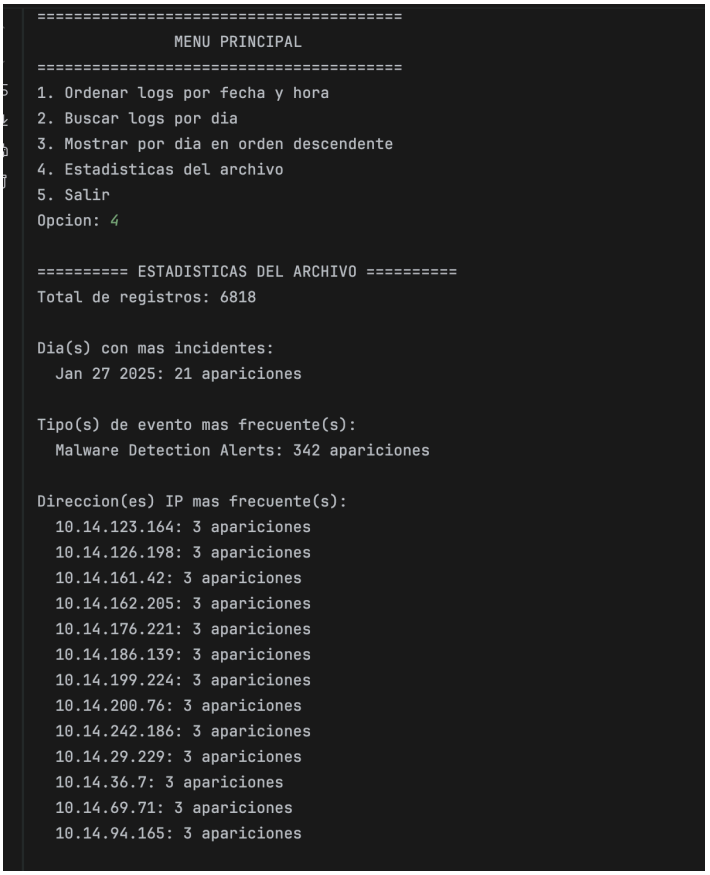
Logs del dia Sep 14 2024:
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Total encontrados: 12
```

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo (proximamente)
5. Salir
Opcion: 3

Logs del dia Sep 14 2024 en orden descendente:
Sep 14 2024 23:30:59 10.14.41.52 Suspicious IP Addresses
Sep 14 2024 23:03:07 10.14.175.98 Port Scanning Activity
Sep 14 2024 22:23:31 10.14.141.44 Failed Login Attempts
Sep 14 2024 20:00:03 10.14.47.235 Data Exfiltration Attempts
Sep 14 2024 18:34:13 10.14.65.141 Anomalous User Activity
Sep 14 2024 11:57:05 10.14.197.208 Cross-Site Scripting
Sep 14 2024 05:15:49 10.14.58.71 Port Scanning Activity
Sep 14 2024 05:14:25 10.14.85.112 Data Exfiltration Attempts
Sep 14 2024 01:18:02 10.14.66.138 Anomalous User Activity
Sep 14 2024 00:52:28 10.14.175.238 Security breaches
Sep 14 2024 00:25:20 10.14.177.24 Suspicious IP Addresses
Sep 14 2024 00:24:23 10.14.115.26 Social engineering attacks
Total encontrados: 12
```

Menú y estadísticas





Advertencias y estabilidad

```
========================================
       ANALIZADOR DE LOGS DE RED
========================================

Selecciona el archivo que quieres cargar:
1. log607-1.txt
2. log607-2.txt
3. Escribir otra ruta
0. Salir
Opcion: 1

Archivo: Evidences/Evidence2/data/log607-1.txt
Registros cargados: 6818
Lineas omitidas: 0

Primeros 5 registros (orden de llegada):
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts
Feb 13 2026 17:14:24 10.14.66.141 SQL injection
May 04 2025 15:46:52 10.14.16.232 Weak or easily guessable passwords
Aug 19 2025 13:20:59 10.14.9.243 Suspicious IP Addresses
Aug 07 2025 18:14:47 10.14.56.220 Malware Detection Alerts

Lectura terminada correctamente.

========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 2
Primero debes ordenar los logs con la opcion 1.
```

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 1

Que crecimiento de comparaciones esperas?
1. n log n
2. n^2
Prediccion: 1

========== RESULTADOS DEL ORDENAMIENTO ==========
Algoritmo: Merge Sort (estable)
Archivo: Evidences/Evidence2/data/log607-1.txt
Registros: 6818
Tiempo: 3.683 ms
Comparaciones: 78330
Profundidad maxima de recursion: 14
Prediccion: n log n
Coincidio con el crecimiento teorico del algoritmo: Si
Orden cronologico correcto: Si
Conservo el orden de llegada en fechas iguales: Si
Archivo generado: Evidences/Evidence2/out/output608.out
```

Tecnológico de Monterrey

Casos de fecha inválida y lista ordenada

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 2
Ingresa el dia (ejemplo: Sep 14 2024): Feb 30 2025
La fecha no existe.
```

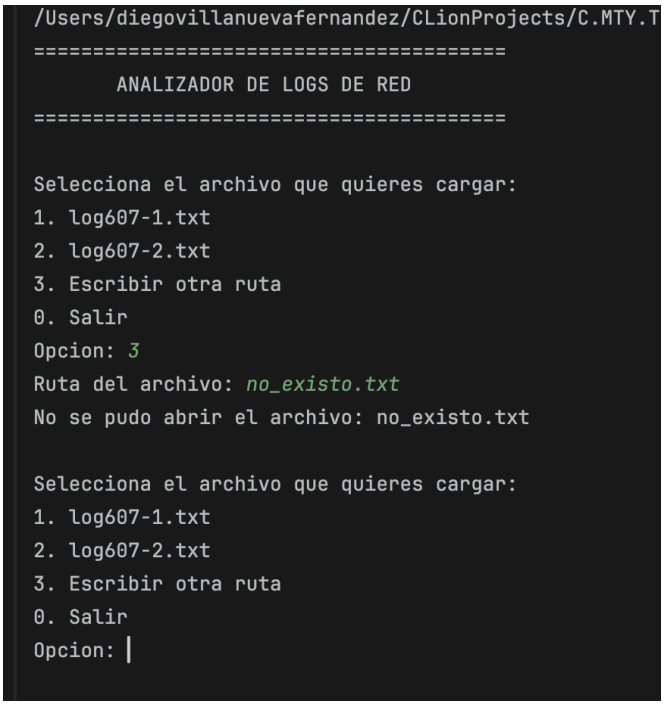
```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 2
Ingresa el dia (ejemplo: Sep 14 2024): Jan 01 2024

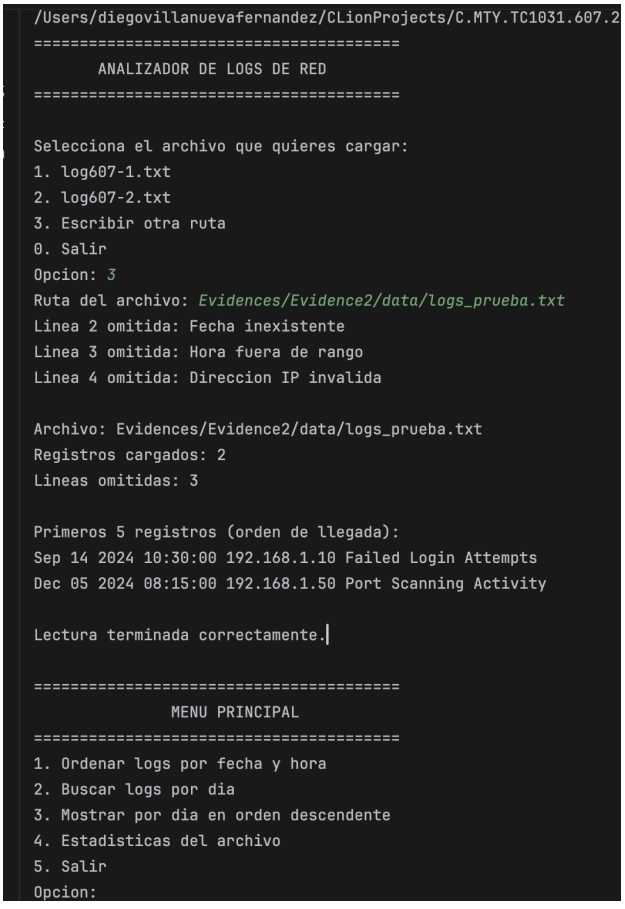
Logs del dia Jan 01 2024:
No se encontraron registros para ese dia.
Total encontrados: 0
```

```
========================================
              MENU PRINCIPAL
========================================
1. Ordenar logs por fecha y hora
2. Buscar logs por dia
3. Mostrar por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Opcion: 1

La lista ya esta ordenada cronologicamente.
Archivo generado: Evidences/Evidence2/out/output608.out
```

Archivo inexistente y líneas mal formadas





Tecnológico de Monterrey

Archivo de salida

```
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication
Sep 08 2024 04:37:37 10.14.248.79 Configuration errors
Sep 08 2024 04:44:15 10.14.135.12 Application error
Sep 08 2024 05:50:57 10.14.22.235 Changes in File Integrity
Sep 08 2024 10:42:42 10.14.40.195 Data corruption
Sep 08 2024 10:47:52 10.14.24.223 Data corruption
Sep 08 2024 15:23:16 10.14.206.243 Weak or easily guessable passwords
Sep 08 2024 15:25:34 10.14.220.55 Broken authentication and session management
Sep 09 2024 01:51:20 10.14.27.106 Application error
Sep 09 2024 04:53:57 10.14.96.200 Suspicious IP Addresses
Sep 09 2024 07:53:44 10.14.230.164 Data Exfiltration Attempts
Sep 09 2024 15:18:08 10.14.182.26 Suspicious IP Addresses
Sep 09 2024 18:59:55 10.14.132.80 Failed Login Attempts
Sep 09 2024 19:17:57 10.14.186.180 Cross-Site Scripting
Sep 10 2024 09:16:12 10.14.245.197 Suspicious IP Addresses
Sep 10 2024 13:50:52 10.14.99.247 Anomalous User Activity
Sep 10 2024 18:14:07 10.14.153.89 Anomalous User Activity
Sep 10 2024 21:18:36 10.14.183.213 Data Exfiltration Attempts
Sep 11 2024 06:44:21 10.14.80.172 Suspicious IP Addresses
Sep 11 2024 08:48:35 10.14.45.122 Anomalous User Activity
Sep 11 2024 13:23:41 10.14.241.4 Changes in File Integrity
Sep 11 2024 17:10:13 10.14.138.230 Encryption Failures
Sep 12 2024 00:25:50 10.14.18.96 Human error
Sep 12 2024 03:30:41 10.14.52.18 SQL injection
Sep 12 2024 03:31:18 10.14.13.16 Changes in File Integrity
Sep 12 2024 03:58:23 10.14.125.11 Encryption Failures
Sep 12 2024 04:03:41 10.14.79.25 Application error
Sep 12 2024 05:48:17 10.14.37.249 Malware Detection Alerts
Sep 12 2024 06:35:19 10.14.211.220 Unauthorized Access Attempts
Sep 12 2024 11:59:08 10.14.226.221 Application error
Sep 12 2024 12:54:23 10.14.252.43 Security breaches
Sep 12 2024 14:37:01 10.14.55.63 Security breaches
Sep 12 2024 16:27:42 10.14.54.112 Load balancing issues
Sep 12 2024 19:11:50 10.14.165.115 Unauthorized Access Attempts
Sep 12 2024 21:23:06 10.14.17.242 Cross-Site Scripting
Sep 12 2024 21:24:52 10.14.51.28 Configuration errors
```

Tecnológico de Monterrey

Resultados del ordenamiento

ARCHIVO	REGISTROS	PREDICCIÓN	TIEMPO	COMPARACIONES	PROFUNDIDAD
log607-1	6,818	O(n log n)	2.915 ms	78,330	14
log607-2	6,818	O(n log n)	1.911 ms	50,624	14

Ambas ejecuciones terminaron con los registros en orden cronológico. La profundidad máxima fue 14 en los dos casos. El segundo archivo, casi ordenado, requirió menos comparaciones y en esta corrida registró menor tiempo. El tiempo puede cambiar según las condiciones del equipo. Para ambos archivos se eligió la predicción n log n, consistente con la complejidad teórica de Merge Sort. La implementación selecciona primero el nodo de la mitad izquierda cuando las fechas y horas son iguales. En la ejecución del primer archivo, el programa también reportó que conservó el orden de llegada de estos registros. Las capturas originales de los resultados aparecen en el anexo.

Prompts utilizados

Primero:
“Analiza solamente las instrucciones y mi repo actual, de lo que tengo ahorita que no ocupo volver a hacer y créame una todo list paso a paso para ir haciendo la evidencia de manera cronológica y organizada”

Me ayudó a identificar archivos existentes que podría reutilizar y a dividir el proyecto en avances. Adapté mi LinkedList para usar head y tail y así poder insertar al final sin recorrer toda la lista, y también revisé el funcionamiento con un programa de pruebas.

Segundo:
“Clangd: In template: ’this’ argument to member function ’getData’ has type ’const LinkedList<int>’, but function is not marked const”

Durante las pruebas de Stack y Queue me apareció este error, y ya me explicó que tenía que poner getData como const para que pudiera llamarse desde métodos const.

Tercero:
“duplicate symbol ’_main’ in: ... TestStructures.cpp.o ... DoublyLinkedList.cpp.o”

Al intentar compilar mi código, el Cmake estaba intentando enlazar dos archivos con main en un mismo ejecutable y ya Chat me dio la explicación inicial, pero no me quedó clara. Entonces tuve que ver un video de YouTube que explicara las CmakeLists y cómo funcionan y cómo las tenemos que declarar y cómo poder agregar dos ejecutables separados. Ya finalmente configuré los targets de la actividad anterior, TestStructurs y Evidence2, por separado.

Mi problema más claro con la IA fue que me proponía cambios de CMake sin explicarme de una forma que yo entendiera por qué aparecía el error. El fallo se detectó en el mensaje de compilación, donde se mostraba el símbolo main duplicado. La solución consistió en separar los archivos con main en diferentes ejecutables. También tuve que revisar sugerencias relacionadas con templates y const, porque algunas modificaciones no encajaban directamente con mis clases.
Al final reutilicé mis estructuras previas, configuré y ejecuté pruebas en CLion, revisé la salida y busqué una explicación externa para CMake. La IA propuso adaptaciones, funciones auxiliares, correcciones de código y la organización del trabajo.

Tecnológico de Monterrey

Reflexión personal

Durante esta actividad, considero que la inteligencia artificial me ayudó principalmente a organizarme mejor. Al ser un proyecto bastante grande, una de las mayores dificultades fue saber por dónde empezar y cómo dividir todo el trabajo. Algo que me sirvió mucho fue identificar qué partes del código ya tenía desarrolladas de actividades anteriores, como las listas ligadas, pilas y filas, para no tener que comenzar desde cero. Esto me permitió ahorrar tiempo y trabajar de manera más ordenada, algo que siento que no aproveché tanto durante la Evidencia 1.

También me di cuenta de que la IA no siempre entiende lo que estoy intentando hacer. Tuve que revisar algunas sugerencias relacionadas con el ordenamiento y con los templates de mis estructuras, porque no siempre se adaptaban a mi código. Para mí, esa fue una lección importante: no basta con copiar lo que propone una herramienta, sino que tengo que verificar que compile, que funcione y que realmente cumpla los requisitos. Sin IA, creo que la actividad habría sido más difícil y lenta; muchas dudas que ahora puedo preguntar directamente antes tenía que buscarlas en Stack Overflow, Reddit o entre varios ejemplos distintos.

Cada estructura tuvo un papel. La lista ligada simple fue la base de la fila y la pila. La fila permitió respetar el orden de llegada de las líneas del archivo, mientras que la lista doblemente ligada guardó los registros y permitió recorrerlos en los dos sentidos. La pila se utilizó para mostrarlos en orden descendente. Sin embargo, creo que esa pila no era estrictamente necesaria: como la lista doblemente ligada ya tiene el apuntador prev, también habría podido recorrerla hacia atrás. Eso habría evitado copiar temporalmente los registros encontrados, aunque usar la pila fue útil para aplicar el comportamiento LIFO visto en clase.

Decidí utilizar Merge Sort en lugar de Quick Sort porque entendí mejor cómo funcionaba y porque garantiza O(n log n) incluso en el peor caso, mientras que Quick Sort puede llegar a $O(n^2)$. Además, Merge Sort encaja bien con listas ligadas porque no necesita consultar elementos por índice; en nuestro programa pudimos reorganizar nodos por sus apuntadores. También mantiene el orden de llegada cuando dos eventos tienen exactamente el mismo timestamp.

Entendí que una búsqueda binaria no tiene la misma ventaja sobre una lista ligada que sobre un arreglo. En un arreglo se puede acceder directamente al elemento central, pero en una lista hay que recorrer los nodos para llegar a él. Repetir esos recorridos hace que se pierda la eficiencia O(log n) y que el costo pueda llegar a O(n). Una alternativa sería crear un arreglo de apuntadores a los nodos ya

ordenados; eso permitiría buscar de forma binaria, pero requeriría memoria adicional O(n) para el índice.

Al comparar ambos archivos, observé que Merge Sort realizó 78,330 comparaciones en el archivo desordenado y 50,624 en el casi ordenado. En estas pruebas, el segundo también tardó menos. Creo que esto ocurrió porque, al unir mitades que ya están más cerca del orden correcto, pueden necesitarse menos comparaciones. En cambio, la profundidad máxima fue 14 en los dos casos, porque el algoritmo siempre divide los datos en mitades independientemente del orden de entrada. Esta profundidad está relacionada con la pila de llamadas recursivas: cada llamada ocupa espacio hasta que regresa. La profundidad fue igual, aunque el trabajo concreto de combinar las mitades cambiara.

Lo que haría diferente sería revisar desde el principio qué código ya tengo listo y entender antes la configuración del proyecto. El caso de CMake me enseñó que, si la explicación de la IA no me sirve, vale la pena buscar otra fuente, como un video, en lugar de seguir haciendo cambios sin entenderlos. En general, aprendí tanto sobre las estructuras de datos y su rendimiento como sobre la importancia de comprobar lo que una IA sugiere.